Escola de Artes, Ciências e Humanidades — Universidade de São Paulo

RELATÓRIO DE CONFIGURAÇÃO DO AMBIENTE

Ambiente de Testes de Injeção de Falhas em Dispositivos IoT

Área: Sistemas de Informação / Segurança da Informação
Estudante: Eduardo Diniz da Silva
Orientador: Prof. Fábio Nakano

1 Resumo

Este relatório descreve a configuração do ambiente de testes de injeção de falhas em dispositivos IoT.

2 Configuração do QEMU

A primeira etapa foi a instalação do QEMU, emulador de hardware, do GDB, depurador de código, do Newlib, biblioteca C padrão e do Make, ferramenta de automação de compilação.

```
sudo dnf install qemu-system-arm
sudo dnf install gdb
sudo dnf install arm-none-eabi-gcc arm-none-eabi-newlib
sudo dnf install make
```

Em seguida, foi feito o mapeamento físico da placa virtual no arquivo `linker.ld`, que define a memória FLASH e RAM, bem como o endereço do topo da pilha (stack) da RAM. Após isso, foi implementado o arquivo `startup.c`, que contém o código de inicialização do sistema. Ambos tiveram como referência o repositório (CORTEXM, 2026) e estão disponíveis no repositório do projeto https://github.com/D1n7z/ic-ascon. Além disso, foi criado o arquivo `Makefile`, que automatiza o processo de compilação do projeto:

- `make build`: compila o projeto e gera o binário`firmware.elf`.
- `make run`: executa o binário no QEMU
- `make clean`: remove os arquivos gerados durante a compilação para uma reconstrução limpa.

Finalizando a configuração inicial do ambiente, foi feito um teste de execução do binário no QEMU, através de um arquivo `main.c`

```c
#include <stdio.h>

extern void initialise_monitor_handles(void);

int main(void) {
    initialise_monitor_handles();

```

```
8           printf("\nHello, World!\n");
9           return 0;
10      }
```

que imprimiu a mensagem "Hello, World!"no console do terminal. Com isso, a parte inicial de instalação e configuração do QEMU foi concluída com sucesso.

3 Configuração do GDB

A próxima etapa foi a configuração do GDB, que é o depurador de código. Para isso, foi criado um novo `main.c` com um código simples de soma para testar o GDB:

```
1       #include <stdio.h>
2
3       extern void initialise_monitor_handles(void);
4
5       int main(void) {
6           initialise_monitor_handles();
7
8           volatile int soma = 0;
9
10          printf("Inicio da execucao do programa\n");
11          for (int i = 0; i < 5; i++) {
12              soma += i;
13          }
14          printf("Soma: %d\n", soma);
15
16          return 0;
17      }
```

O código foi compilado e executado normalmente no QEMU, imprimindo a mensagem "Inicio da execução do programa"e o resultado da soma = 10. Em seguida, foi executado no QEMU com o comando `qemu-system-arm -M mps2-an385 -cpu cortex-m3 -m 16M -nographic -kernel firmware.elf -semihosting -S -gdb tcp::1234` e o GDB foi iniciado com o comando `gdb firmware.elf` e conectado ao QEMU com o comando `target remote :1234`. Atrvés do GDB, foi possível depurar o código e durante a execução do programa, um registrador teve seu valor alterado de 0 para 100, e o resultado da soma foi alterado de 10 para 110. A fim de validar o ambiente, o processo foi repetido com um script em Python:

```
1       import gdb
2
3       gdb.execute("set pagination off")
4
5       print("Injetor de falhas iniciado.")
6
7       # Conectar ao QEMU
8       gdb.execute("target remote :1234")
9
10      # Definir breakpoint no início de main
```

```
    gdb.execute("break main")
    gdb.execute("continue")

    # Pula para o início do laço for
    for _ in range(3):
        gdb.execute("next")

    # Entrar no laço for
    for _ in range(8):
        gdb.execute("stepi")

    # Lê o valor do registrador r3
    valor_r3 = gdb.parse_and_eval("$r3")
    print(f"Valor original do registrador r3: {valor_r3}")

    # Altera o valor do registrador r3 para 100
    print("Alterando o valor do registrador r3 para 100.")
    gdb.execute("set $r3 = 100")

    # Continua a execução do programa
    print("Continuando a execução do programa")
    gdb.execute("continue")

    # Desconecta do GDB e encerra o script
    gdb.execute("quit")
```

O resultado da execução do script foi o mesmo que a execução manual, alterando o valor do registrador r3 de 0 para 100 e o resultado da soma de 10 para 110. Depois foi realizado mais um teste, alterando o valor do registrador pc para 0, que gerou um HardFault, que é uma exceção gerada pelo processador quando ocorre um erro crítico.

```
    Início da execução do programa
    qemu: fatal: Lockup: can't escalate 3 to HardFault (current priority -1

    R00=00000000 R01=20000894 R02=00000000 R03=20000808
    R04=00000000 R05=00000000 R06=00000000 R07=203fffe8
    R08=00000000 R09=00000000 R10=00000000 R11=00000000
    R12=00000001 R13=203ffff8 R14=ffffffff R15=d0024282
    XPSR=61000003 -ZC- T handler
    FPSCR: 00000000
    [1]    103042 IOT instruction (core dumped)  qemu-system-arm -M mps2-an
-S
```

O HardFault foi capturado pelo QEMU e exibido no console do terminal, indicando que o ambiente de testes de injeção de falhas em dispositivos IoT está funcionando corretamente. Com isso, a configuração do GDB foi concluída com sucesso.

Referências Bibliográficas

CORTEXM. **baremetal: ARM Cortex-M bare-metal example code in C++ and CMake build**. [S.l.]: GitHub, 2026. Disponível em: <https://github.com/cortexm/baremetal>. Acesso em: 12 out. 2026.